

深圳大学考试答题纸

(以论文、报告等形式考核专用)
二〇二四~二〇二五 学年度第 二 学期

课程编号	1502880001	课程名称	基于 Web 的编程	主讲教师	Basker George	评分	
学号	2022155033	姓名	洪子敬	专业年级	2022 级软件工程 (腾班)		
学号	2022150102	姓名	温奇伟	专业年级	2022 级计算机科学与技术		
学号	2022150120	姓名	麦子垣	专业年级	2022 级计算机科学与技术		

教师评语: The mark obtained for different categories as per the grading policy are as follows

序号	评分点 (分值)	成绩
1	功能实现完整性: 页面结构、模块功能是否齐全且运行良好 (40 分)	
2	用户体验与交互设计: 页面交互流程是否清晰合理、易用 (10 分)	
3	视觉美感与细节: 页面设计统一、美观、有明确风格 (5 分)	
4	性能与适配设计: 兼容多浏览器、适配不同屏幕的能力 (5 分)	
5	团队协作与贡献: 分工清晰、每人独立完成指定模块 (5 分)	
6	功能创新与亮点: 新颖设计、互动体验、个性功能 (5 分)	
7	整体分析与平台设计: 项目目标、用户流程、技术选型等 (5 分)	
8	个人模块设计与实现: 具体 HTML、CSS、JS 设计分析与实现细节 (20 分)	
9	项目总结与反思: 项目协作过程体会、改进建议等 (5 分)	
合计	100	



Basker George
30 June 2025

题目： 校园交友平台

组员个人总分权重分配表：

排序	姓名	学号	项目个人权重	个人最后得分	评分人
1	洪子敬	2022155033	100%		
2	温奇炜	2022150102	95%		
3	麦子垣	2022150120	75%		
4					
5					

任务分工及所完成的状况。
(2 人总分为 190% *教师评分, 3 人总分为 270% *教师评分)

个人总分权重分配表

(1 人总分为 100%，2 人为 200%，3 人为 270%，4 人为 350%，5 人为 430%)

排序	姓名	学号	项目个人权重
1	洪子敬	2022155033	100%
2	温奇炜	2022150102	95%
3	麦子垣	2022150120	75%
4			
5			

我组成员总共__3__名，权重总和为：270 %

数据库大作业 Report

一、实验概述

本次大作业聚焦于校园社交网页的开发，致力于打造一个集普通用户、用户系统、动态发布与浏览、互动社交和签到打卡等多功能于一体的校园社交平台 CampusConnect。通过综合运用 HTML、CSS (Tailwind CSS)、JavaScript 等前端技术，结合后端接口实现与本地的数据库进行数据交互，基本按预定设想成功完成了校园社交网页的开发。

二、实验环境

2.1 开发工具

- 文本编辑器：Visual Studio Code
- 浏览器：Google Chrome

2.2 技术栈

- 前端：HTML、CSS (Tailwind CSS)、JavaScript
- 后端：Express、mysql2/promise、bcryptjs、Multer、cors、dotenv

三、网页设计

整体的 CampusConnect 网页设计大致可以分 5 个部分，分别是普通游客板块、用户系统板块、动态发布和浏览板块、互动与社交板块和个性化板块，下面对这些板块进行逐一介绍。

3.1 普通用户板块

整体实现：为了保障游客能有较好的体验，相比登陆后的页面，这里保留了帖子的浏览、评论的浏览和个人主页面的浏览，其他部分包括发布帖子、关注、收藏、私信、点赞等功能全部无法使用，当非登录用户点击时会弹出“请先登录/注册以使用此功能”的警告并弹出登录注册页面给用户进行登录注册。

具体效果：当处于未登录状态时，整体的界面如图 1 所示：

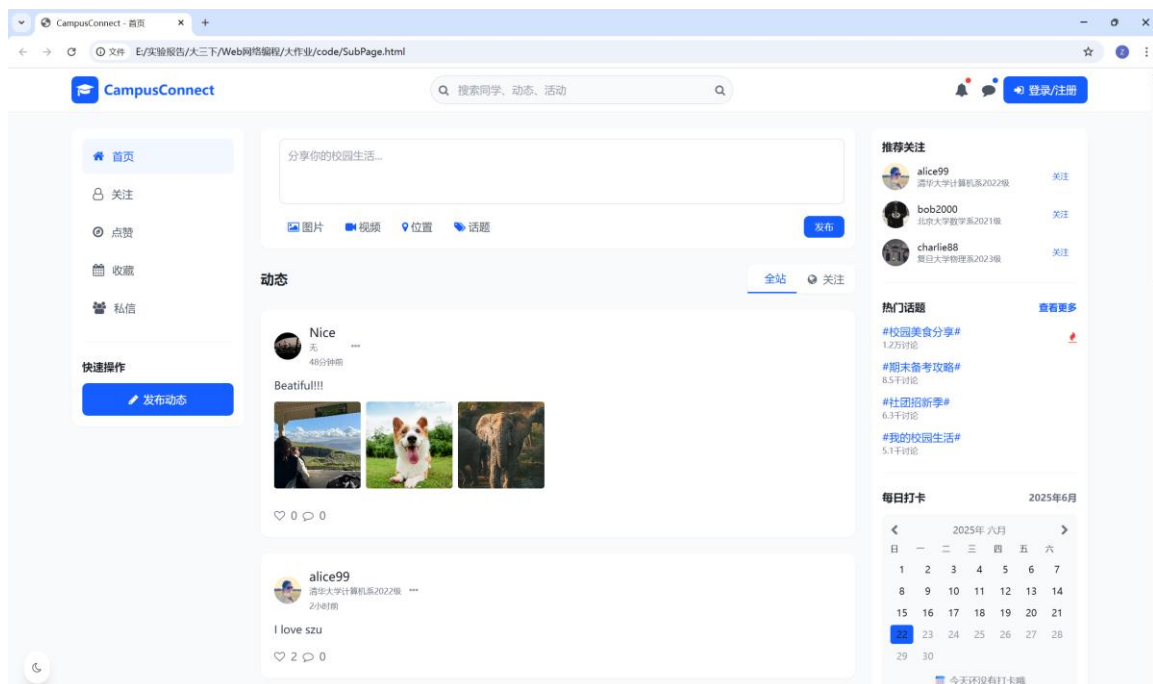


图 1. 未登录网页

3.2 用户系统板块

3.2.1 注册板块

整体实现：未注册用户通过点击图 1 右上角的“登录/注册”按钮进行注册，注册内容分为“上传用户头像”、“昵称”、“邮箱”、“个人简介”、“兴趣标签”、“密码”、“确认密码”和服务条款等内容，用户通过在表单中上传头像（否则会使用默认头像），填写个人信息（个人昵称、邮箱、密码是必须填的，其他选填），具体通过 Javascript 进行表单绑定，将结果通过/api/register 传递给后端 API 将新建用户信息添加到数据库中。

具体效果：完成代码编写，点击注册并填写信息效果如图 2 所示：



图 2. 注册页面 初始页面（左）填写信息（右）

注意：由于 Web 浏览器自带密码的显示和遮蔽按钮，为了避免重复这里直接去掉了密码的遮蔽和显示功能，直接使用浏览器自带的功能。

HTML 中的密码显示使用不同强度的评估并通过 Javascript 监听进行实时地评估并更新评估结果，其中密码的强度计算逻辑函数如下所示：

```
// 计算密码强度
function calculatePasswordStrength(password) {
  let strength = 0;
  if (password.length >= 8) strength++;
  if (/[a-z]/.test(password)) strength++;
  if (/[A-Z]/.test(password)) strength++;
  if (/[0-9]/.test(password)) strength++;
  if (/^[a-zA-Z0-9]/.test(password)) strength++;
  return strength;
}
```

3.2.2 登录板块

整体实现：与注册功能类似，在登录框中填写用户的邮箱和密码在后端接口中验证成功后即可登录，不过登录板块在通过表框获取到 `FormData` 数据之后，需要将当前用户信息存储到本地 `localStorage` 服务中（虽然存储空间小，但是够用），同时更新全局用户变量的状态，方便后续获取当前用户信息的其他接口使用；通过使用 `localStorage` 和全局用户变量，可以达到刷新网页仍然保留登录状态的效果，具体通过在 `document` 加载完成后进行状态检查，如果 `localStorage` 中有相应的用户信息则进行全局状态更新。

同时，在登录成功后，如图 1 所示，此时的网页是还没有登录成功的，所以相关的用户信息都没有显示出来，所以需要添加相应的 `Javascript` 代码，监听用户状态，当用户登录成功后进行用户信息的更新的，包括“登录/注册”按钮替换为当前用户的信息，右侧添加当前用户的信息栏，评论和发布动态时添加用户头像等。

具体效果：完成登录代码编写后，点击登录并填写信息效果如图 3 所示：



图 3. 登录页面 初始页面（左）填写信息（右）

登录成功后，完成前面提及所有动态元素的实时更新后，可以得到如图 4 所示的效果：

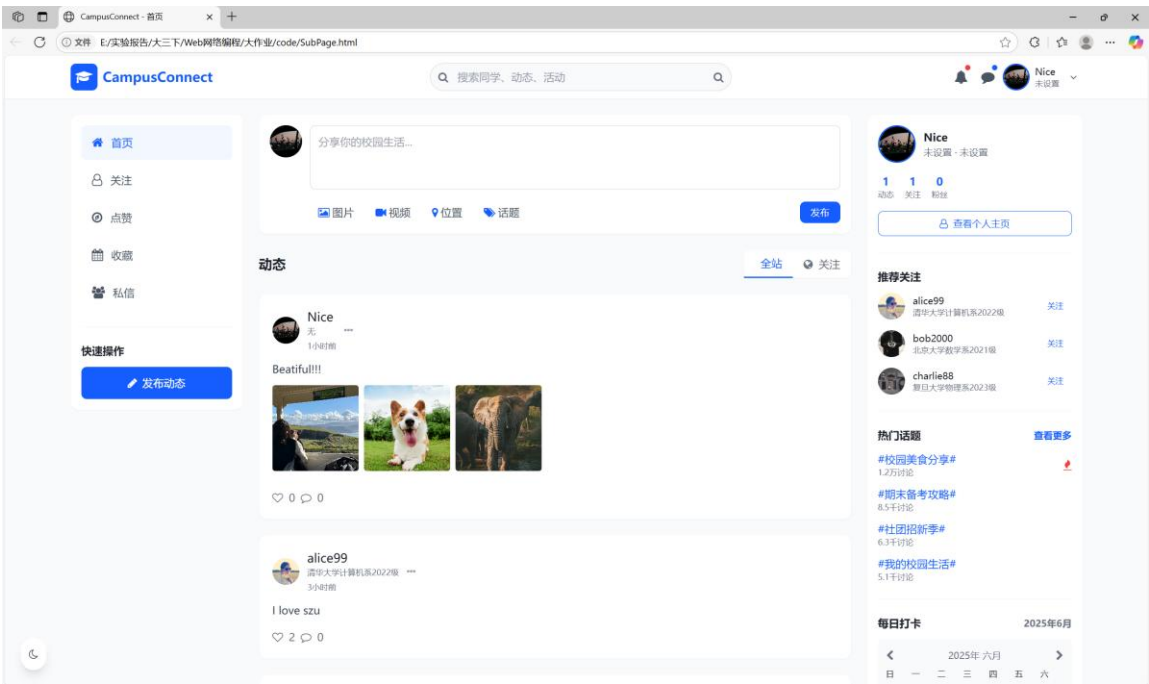


图 4. 登录后的网页

3.2.3 管理员板块

整体实现：由于前面提到我们使用的是全局变量来存储当前用户的信息，所以这里用全局变量中的“type”字段来指代当前用户的身份，0 是普通用户，1 是管理员（当然用户表也是如此），当用户是管理员时这里并没有给与管理员封禁用户的权限，而是给与删除评论的功能而已，如图 5 所示，点击动态帖子右边的菜单选项，如果非帖子的主人一般来说是不会显示删除按钮给删除的，但是这里只要检测到当前用户是管理员，则点开菜单栏之后都会有删除功能。

具体效果：分别登录不同权限账户，可以看到如图 5 一样不同的菜单显示：

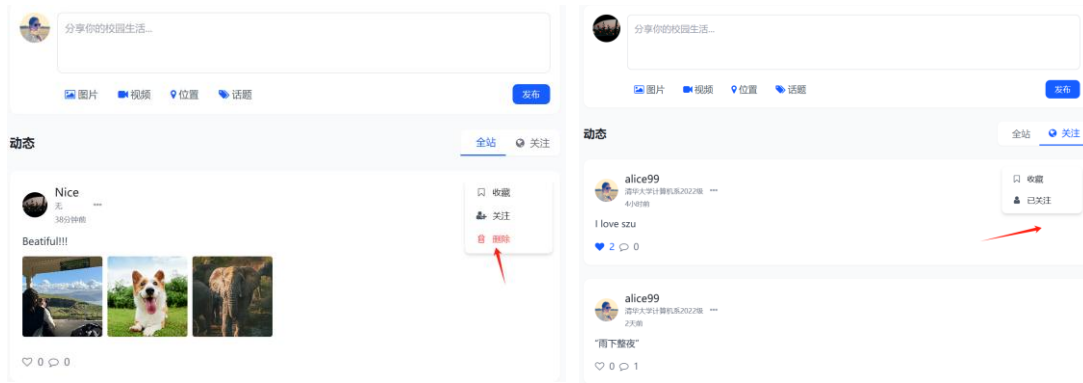


图 5. 管理员用户效果（左）普通用户（右）

3.2.4 个人主页功能（包括编辑个人信息和帖子）

整体实现：为了整体美观，这里在主 HTML 基础上添加了独立的个人主页 HTML，前面提到每个帖子存储着当前帖子的用户 ID，所以通过 a 标签的长链接跳转时则可以从后端根据此用户 ID 找到相应的用户信息，并通过 Javascript 补充相应的 HTML 内容，从而将一个用户页面渲染出来；此外，还需给用户提供编辑功能，在通过判断当前用户与帖子的用户是同一个时，则会显示“编辑个人资料”的按钮，为当前用户提供编辑个人信息的机会。当然除此之外，当前的登录用户还能通过右上角的下拉菜单中的人型图标，或是右侧栏的“查看个人主页”按钮进入，如图 6 所示，当前用户有多种进入个人主页的方式。

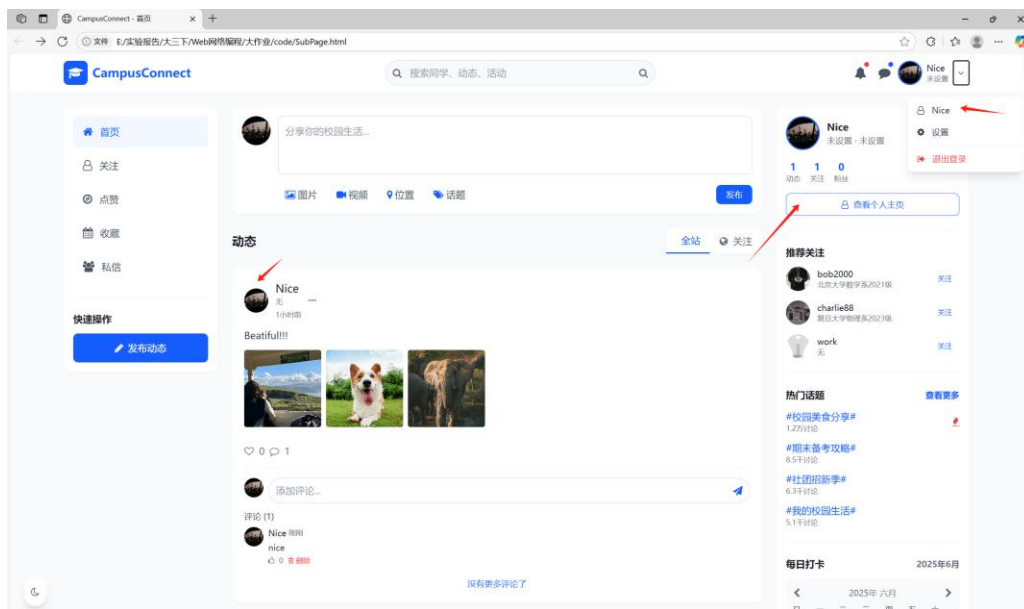


图 6. 多种进入个人主页的方式

具体效果：如果当前登录用户点击其他用户的头像时会进入其他用户的个人主页，此时的效果如图 7 所示；如果当前登录用户通过图 6 多种方式进去个人主页后，如图 8 所示，中间还会额外的显示一个“编辑个人资料”的按钮，点击该按钮会弹出一个编辑个人资料的模式框，通过修改上面的个人内容可以直接对数据库进行修改。

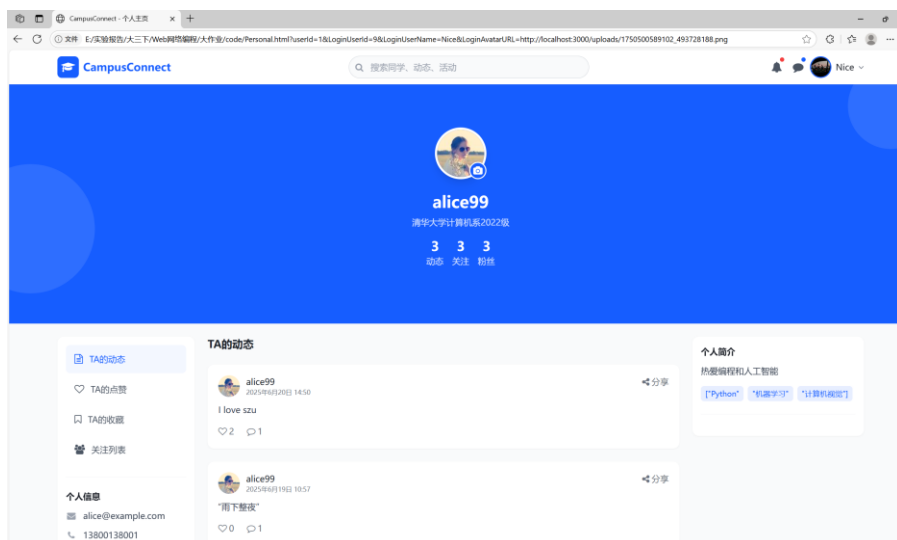


图 7. 非登录用户的个人主页

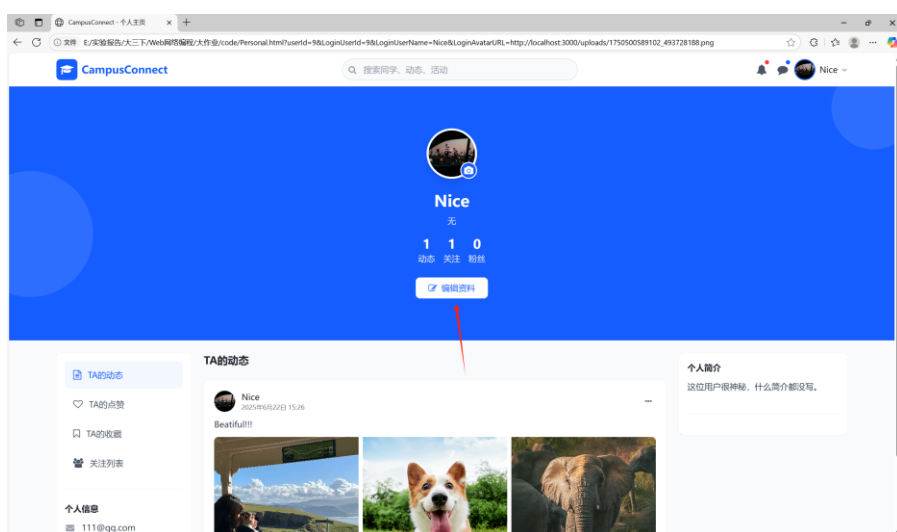


图 8. 登陆用户的个人主页

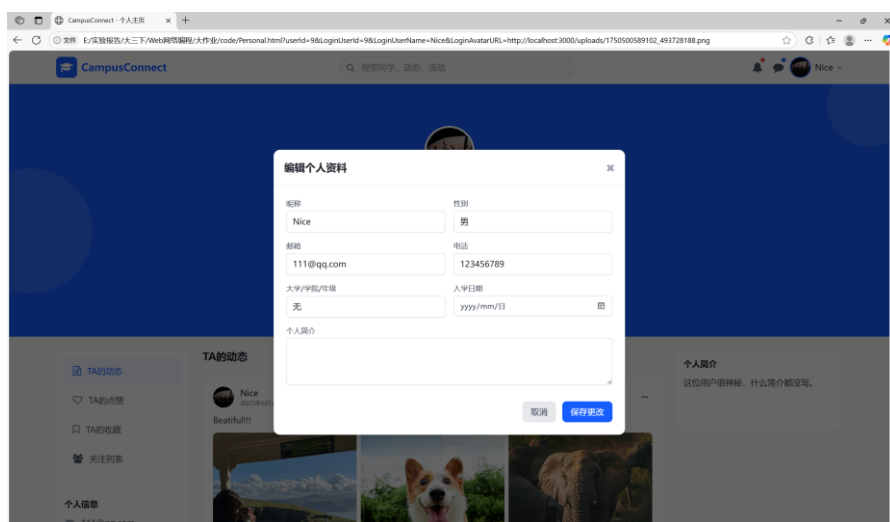


图 9. 编辑个人资料模态框

当然通过图 7 图 8 对比不难发现，如果当前用户时登录用户时，帖子右上角从分享按钮变成了三个点的菜单，通过点击该菜单可以实现对帖子的编辑（文字的修改和新图片的上传）和删除功能，具体也是绑定后端的 API 实现，具体效果如图 10 所示：

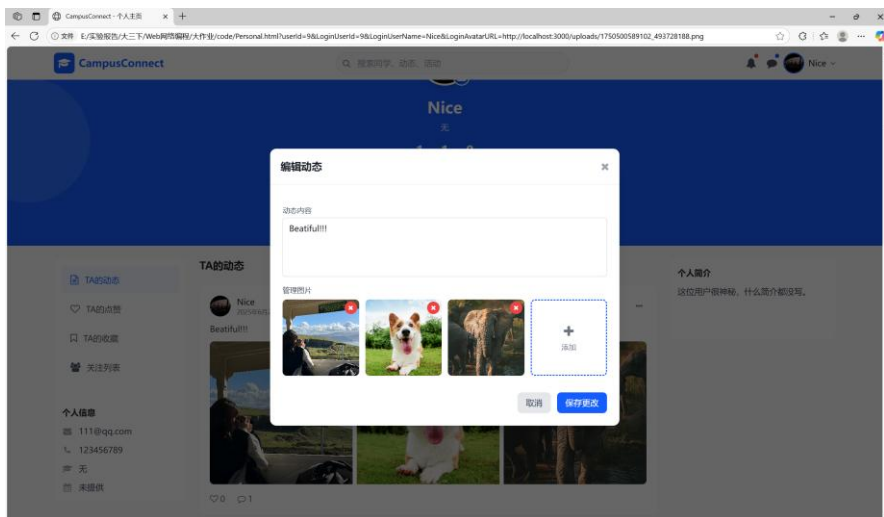


图 10. 编辑动态模态框

3.3 动态发布与浏览模块

3.3.1 动态发布板块

整体实现：如图 1 所示，动态发布有两个地方，分别是左侧的快捷操作和中间上方的发布模框，为了简化操作，这里直接进行统一化操作，点击任何一个地方都弹出来一个模态框，通过绑定模态框上的文字框和图片上传框进行数据的打包；不过这里值得注意的是需要对图片数据做一次转变，具体通过编写后端接口，将当前图片存储到特定路径下，同时将当前服务器路径+相对路径作为图片数据，方便统一对图片数据进行操作。当帖子发布成功后，也就是帖子成功存储在数据库之后，这时会重新渲染主页中心的动态流，使得实时性更强些。

具体效果：如图 11 所示，可以在其中填写帖子的文字信息以及上传一个或者多张图片，位置和话题由于时间关系这里并未加以实现，点击“发布”即可发布动态。



图 11. 动态发布模态框 初始页面（左）填写信息（右）

3.3.2 动态点赞功能

整体实现：通过在数据库中添加相应的点赞表，并为每个动态帖子添加 like_count 属性，用于实时的点赞效果展示（当前帖子的点赞数），同时需要为其绑定相应的点击事件。当前登录用户为该帖子点赞时，调用后端的接口/api/like 更新点赞表和帖子表，并将当前的空心图标替换为实心图标代表点赞成功；当取消点赞时，调用后端/api/unlike 更新点赞表和帖子表，并将当前的实心图标替换为空心图标代表取消点赞成功。

具体效果：完成点赞功能实现后，点赞前后的效果如图 12 所示：

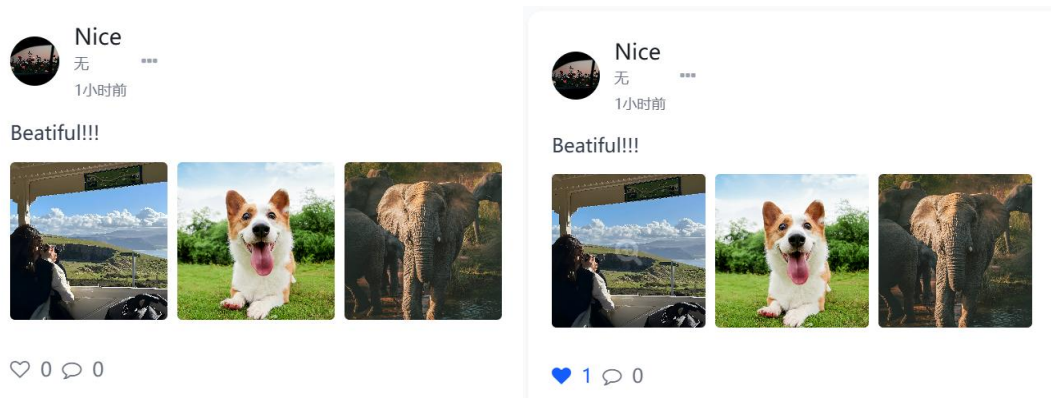


图 12. 点赞前（左）点赞后（右）

3.3.3 评论功能（发布评论+评论点赞）

整体实现：为了实现评论功能，这里为其添加了一个模态框，绑定图标的点击事件，当用户点击挂机评论区图标时会从后端/api/comments 中加载当前帖子对应的评论（通过渲染帖子时存储的帖子ID 从而能正确识别是哪个帖子的评论），此时不仅需要渲染评论，还需要渲染当前用户的头像和用户名代表是哪个用户发布的评论；此外，还添加了一个评论点赞功能，这个功能与前面的动态点赞功能类似，不过这里是对评论添加 like_count 属性，需要额外增加一个评论点赞表用于存储用户与评论的关系表。

具体实现：完成了前面的代码编写后，点击评论图标并发布评论效果如图 13 和图 14 所示：



图 13. 点击评论区图标效果

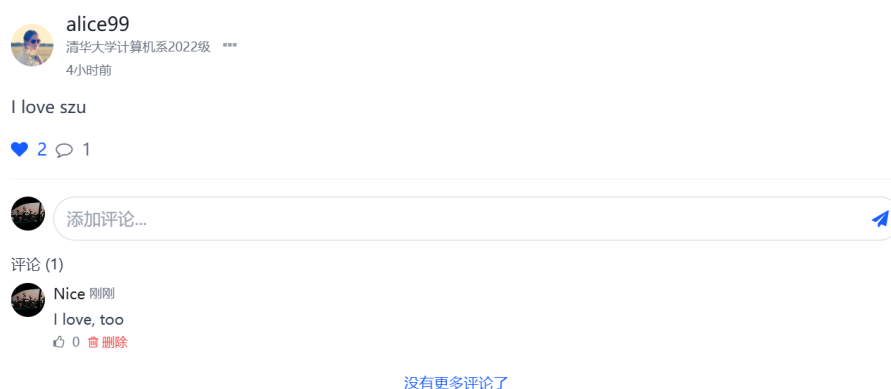


图 14 发布评论效果

同理，为评论点赞可以得到如图 15 所示的效果，为了和帖子点赞混淆，这里将图标从原来的爱心改为了大拇指。



图 15. 评论点赞效果

3.3.4 删除功能

整体实现：既然能发布，这里也要为其添加删除功能，对于帖子，正如前面所说，点击帖子旁边的菜单，如果当前权限满足（个人/管理员），则会显示“删除”按钮供用户点击后删除，此时删除帖子会删除相应的评论以及评论点赞，并且更新数据库中相应的用户的相关信息；对于评论的删除，与帖子不同，删除后不仅要更新用户表，还要更新帖子表的信息。

具体效果：添加删除功能后的帖子和评论的删除按钮如图 16 所示：

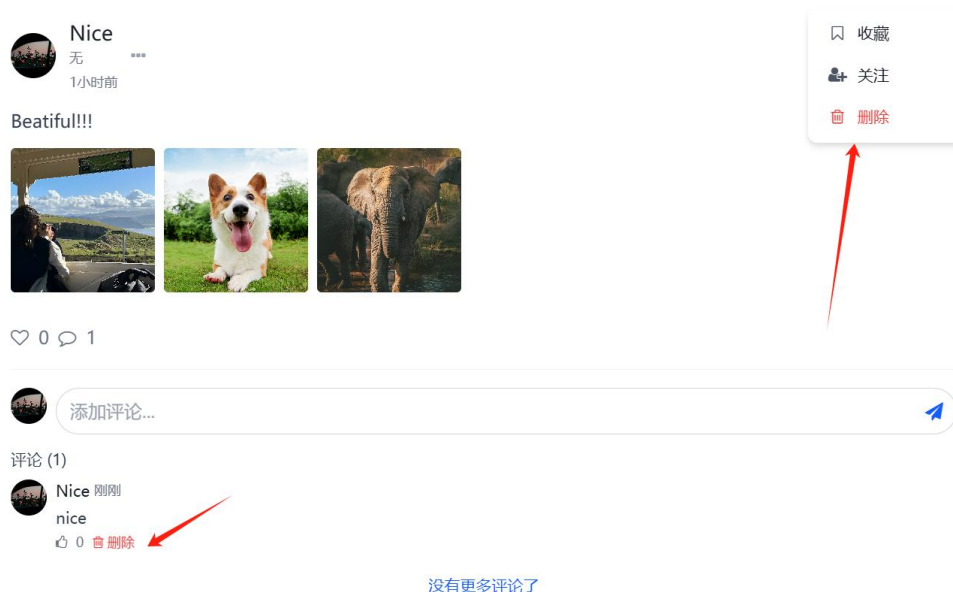


图 16. 删除图标效果

3.4 互动与社交模块

3.4.1 关注功能

整体实现：为了更好的促进用户的交流，为用户提供关注功能，用户可以通过点击每个帖子的“关注”按钮即可关注当前帖子的用户，关注后也可以通过再次点击菜单相同位置的“取消关注”取消关注当前用户；通过为每个用户表添加一个粉丝数 `follower_count` 和关注数 `following_count`，同时创建一个新的关注表，关联每组联系的用户，通过此表可以统计前面提及的两个数量；而粉丝数和关注数会在用户登录成功后显示在右侧栏上，同时每个用户的个人主页同样会显示这两个数量。

具体效果：如图 17 所示，点击当前帖子的菜单按钮，点击“关注”此时图标会从原来的图标变为绿色的无加号图标代表以及关注，当然关注后粉丝量和关注数量并没有设置实时展示，而是下次登录时才会更新，这也不会影响实际的效果。

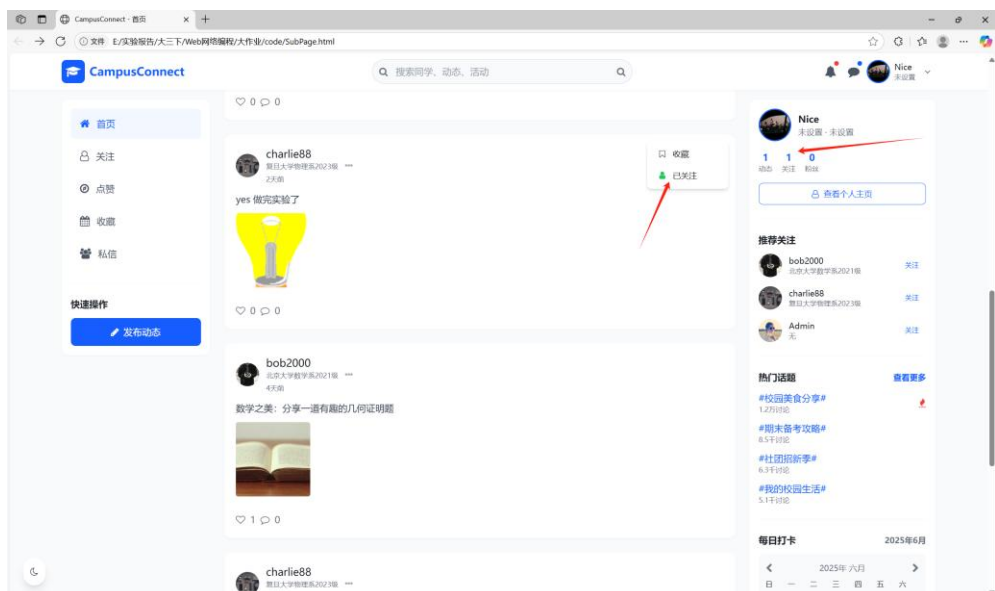


图 17. 关注效果

同时，为了方便用于当前用户查看自己的关注对象，在左侧添加一个功能按钮“关注”，通过点击该按钮会在后台查询后返回结果显示在模态框中，用户可以在模态框对用户进行取消关注。效果如图 18 所示：

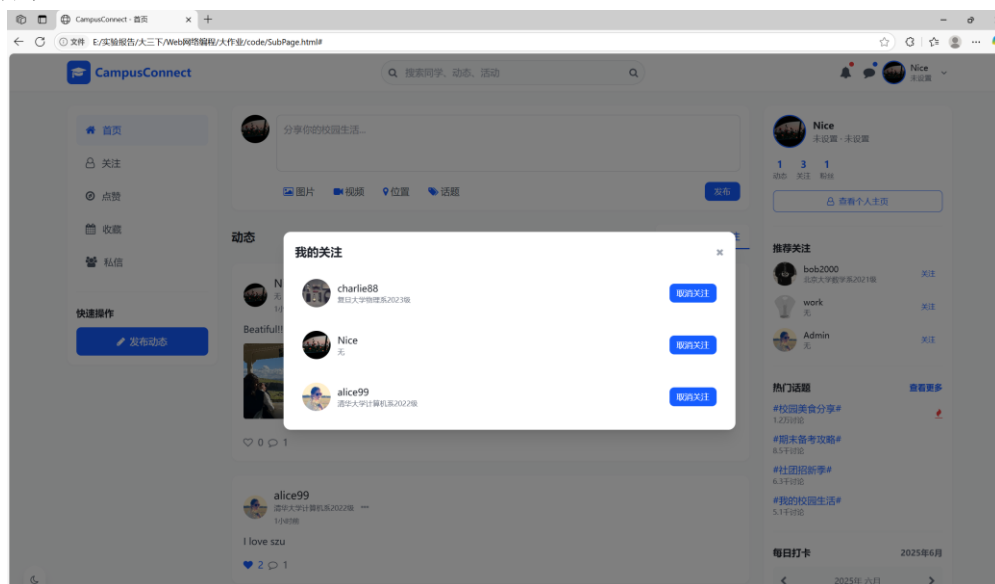


图 18. 当前用户关注列表

3.4.2 首页动态和全站动态切换

整体实现：在中间的帖子流中，添加一个切换按钮，通过点击该按钮可以进行“全站动态”和“关注动态”的切换，实际实现也特别简单，在原来的基础上多添加一个监听，如果切换了两者中的一种就重新渲染帖子流，但两者也有区别，需要额外编写一个接口给“关注动态”，只筛选当前用户的关注对象的帖子进行渲染，其他地方基本一致。

具体效果：网站默认从“全站动态”进入，当点击“关注”时则会切换帖子渲染，如图 19 所示：

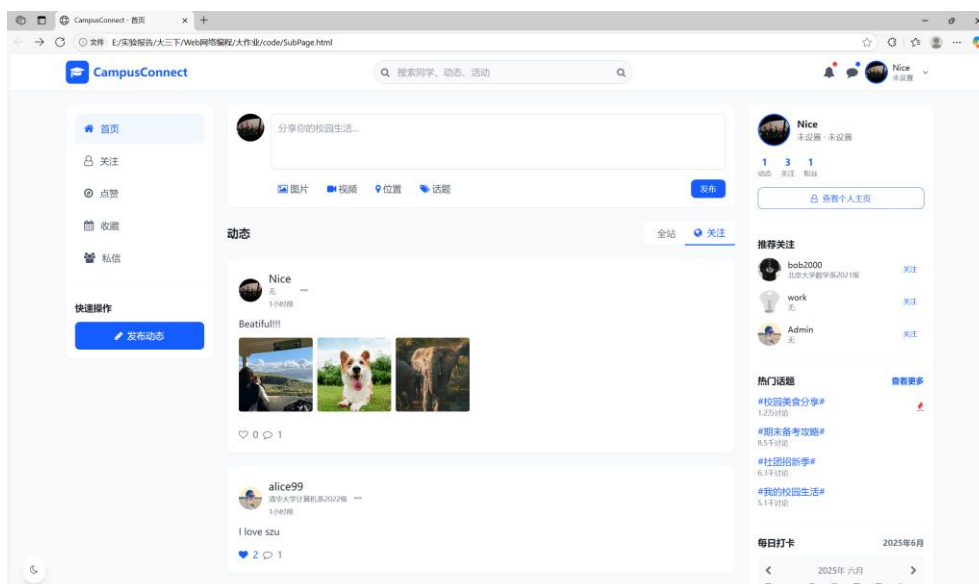


图 19. 关注动态切换

3.4.3 私信功能实现

整体实现：由于本版块只需要界面实现，所以这里直接渲染提交已经好的对话记录，但是添加一点可以交互的东西就是用户可以在多个对话框之间来回切换，其他均为静态的，实现也比较简单，通过对一个模态框进行事件绑定即可；此外，主界面上通过绑定左侧边栏的“私信”按钮，通过点击按钮会触发相应的事件，从而显示私信对话框。

具体效果：点击左侧边栏的“私信”按钮，显示效果如图 20 所示：

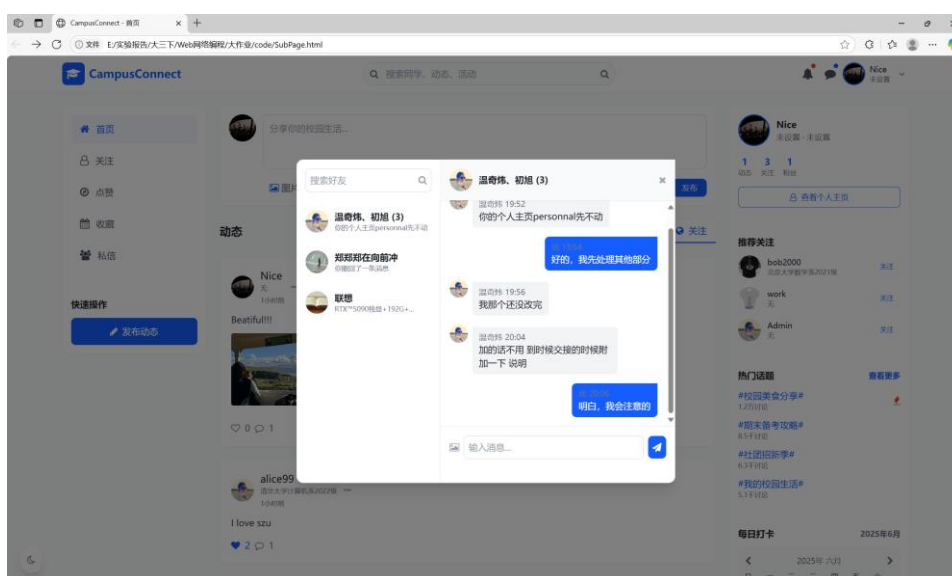


图 20. 私信效果

3.4.4 主题切换（夜间模式）

整体实现：主题的设置也挺简单的，通过将原来的表单栏、标题栏和导航栏等设置为另外一个色系即可达到不同主题的效果，这里也是简单地切换了黑灰色系（原来是白灰），没有其他更改。只有按钮绑定在左下角的一个月亮形状的按钮，通过绑定事件来实现颜色切换。

具体效果：点击左下角的切换夜间模式按钮，达到如图 21 所示的效果：

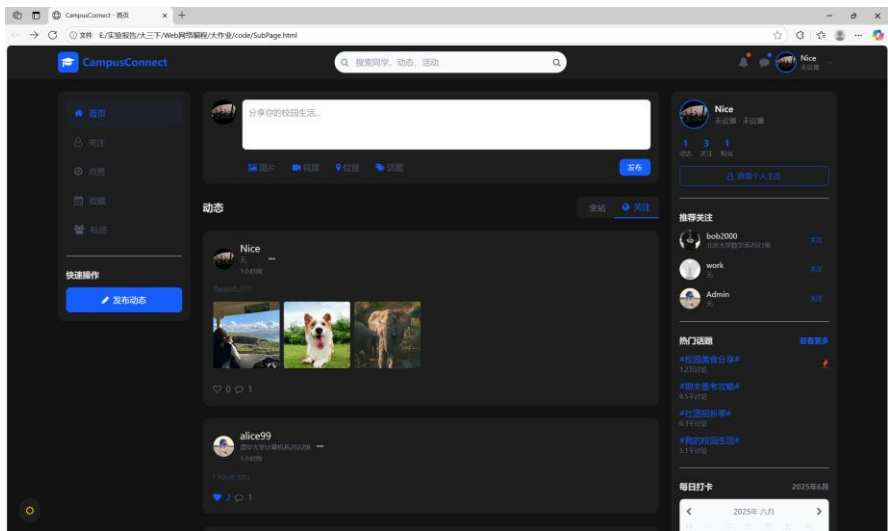


图 21. 夜间模式效果

3.4.5 收藏功能

整体实现：与之前点赞逻辑类似，这里也是在每个帖子的菜单中添加此功能按钮，同时需要在数据库中多建一张收藏表，通过点击收藏，将相应的帖子内容和当前收藏用户进行关联；原本是未填充的星号，当用户点击收藏后与之前类似，这里进行了绿色填充，符合日常用法。

具体效果：点击帖子的收藏按钮，效果如图 22 所示：

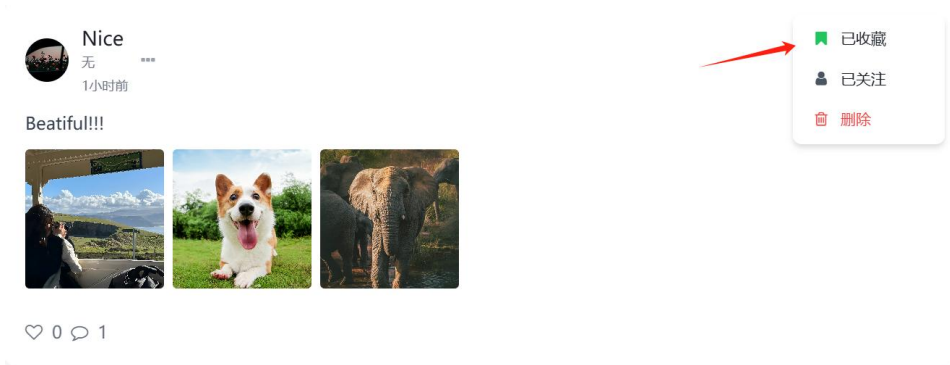


图 22. 帖子收藏效果

除此之外，与前面关注功能类似，为了方便用户查看当前自己收藏的内容，同样地设置左侧栏的“收藏”按钮，用户点击后可以显示当前用户的收藏列表，同时也绑定了相应的事件响应，用户可以直接点击“取消收藏”来取消该帖子的收藏，具体效果如图 23 所示：

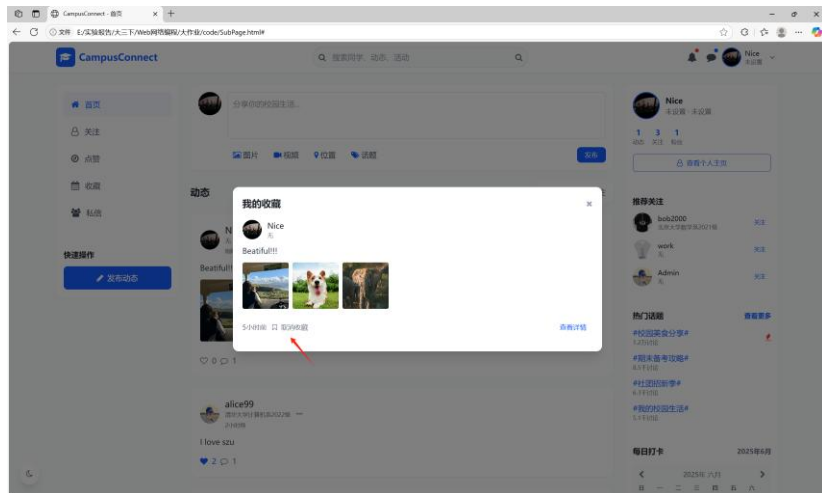


图 23. 当前用户收藏帖子

3.5 个性化板块

3.5.1 用户推荐

整体实现：为了促进用户间的交流，这里引入用户推荐板块，不过这里的推荐算法也比较简单，简单地推荐一些发帖子数量高、粉丝数量多的用户，并从这些用户中随机选择三个，获取他们的用户信息，返回到前端进行展示；核心代码如下：

```
const query = `
    SELECT
        u.user_id, u.username, u.avatar_url, u.school_info,
        u.follower_count, u.post_count,
        0 as is_following
    FROM users u
    WHERE u.user_id NOT IN (SELECT following_user_id FROM Follows
        WHERE follower_user_id = ?)
        and u.user_id != ?
    ORDER BY
        u.follower_count DESC,
        u.post_count DESC,
        RAND() -- 随机排序，增加多样性
    LIMIT ? OFFSET ?`;
```

具体效果：当用户登录后右侧会推荐当前用户值得改造农户的对象，效果如图 24（左）所示：点击“关注”后会显示“关注中”，成功后该用户会添加到关注列表，并从推荐列表消失，如图 24（右）所示：

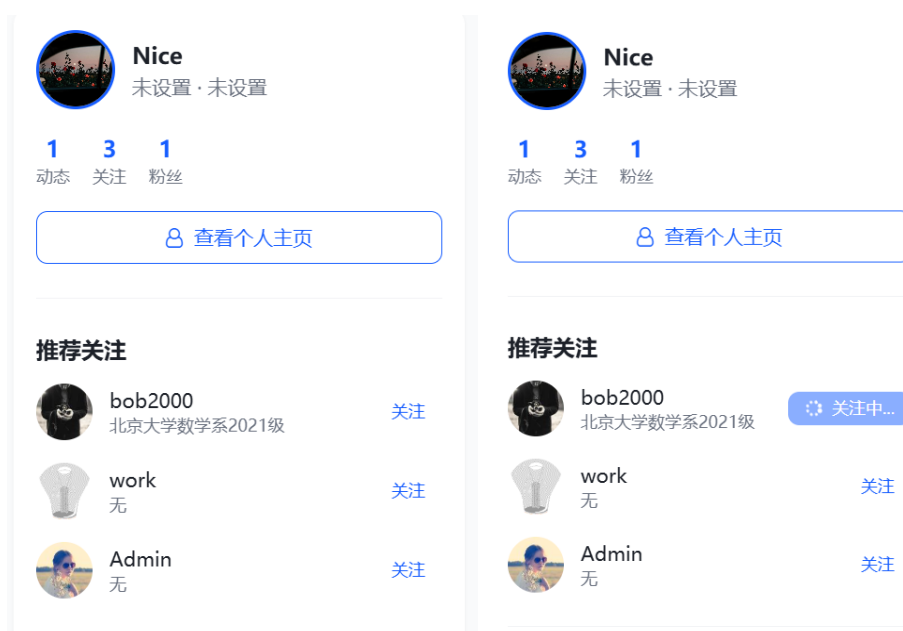


图 24. 用户推荐效果

3.5.2 打卡签到

整体实现：为了促进用户多使用此平台，因此右下角添加了打卡功能；通过显示当前月份的打卡天数，打卡的天添加相应的颜色代表签到成功（借鉴 Leetcode 的签到）；并且设置签到时可以添加当前的心情，点击签到的某一天，上面还会显示当时签到心情，相当于一个小日记的功能，可以吸引年轻的用户来使用，同时签到成功还会提供“太棒了”等显式信息吸引用户打卡。

具体效果： 点击右下角的签到，可以得到如图 25 的效果：

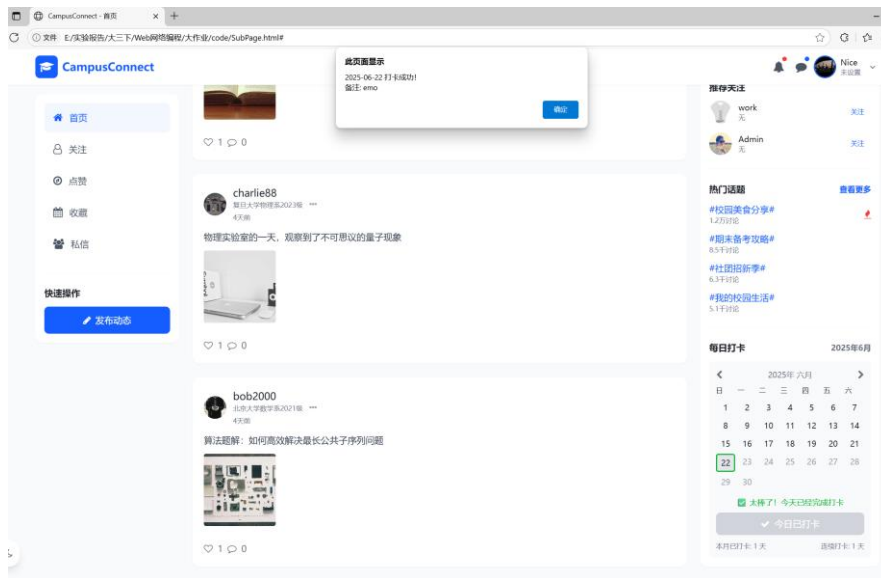


图 25. 打卡效果

3.5.3 搜索功能

整体实现： 增加一个搜索功能，支持搜索名字相近的用户和词条内容包含搜索内容的帖子，实际功能主要由 SQL 语句完成，找到相应用户或者帖子返回，在模态框内渲染相应的内容即可；主要功能语句如下：

```
// 搜索用户`
const userSearchQuery = `
    SELECT user_id, username, avatar_url, school_info, gender
    FROM Users
    WHERE username LIKE ?
    LIMIT 5;
`;

// 搜索动态
const postSearchQuery = `
    SELECT
        p.post_id, p.content, p.media_url, p.created_at, p.like_count, p.comment_count,
        u.user_id AS author_user_id,
        u.username AS author_username,
        u.gender AS author_gender,
        u.avatar_url AS author_avatar_url
    FROM Posts p
    JOIN Users u ON p.user_id = u.user_id
    WHERE p.content LIKE ?
    ORDER BY p.created_at DESC
    LIMIT 10;
`;
```

具体效果： 点击搜索框，搜索关键词“Nice”，返回查询到的是一个用户，如图 26（左）所示，当搜索关键词是“love”，返回查询到是一个帖子，如图 26（右）所示：

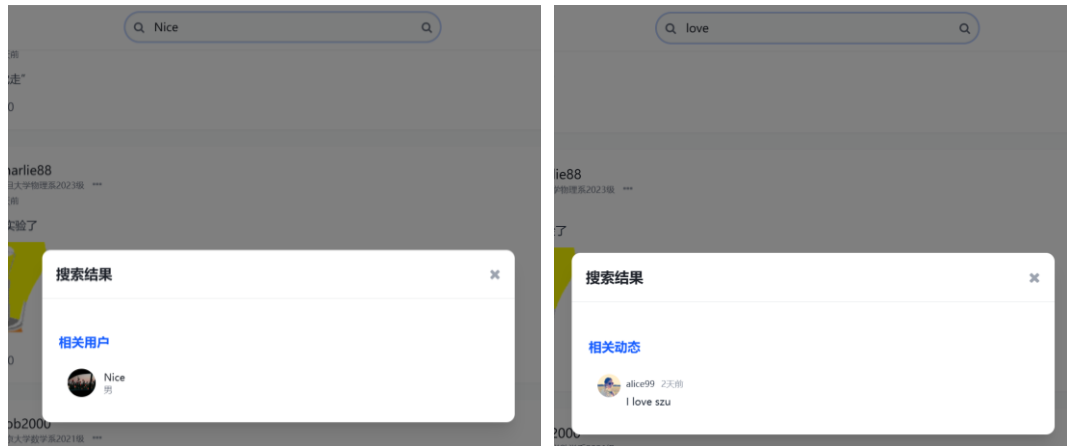


图 26. 搜索结果 1（左）搜索结果 2（右）

四、后端服务设计

本次后端采用基于 Express 框架的 API 开发,通过 mysql2/promise 模块连接本地 MySQL 数据库,通过 Multer 处理文件上传,使用 bcryptjs 来进行密码加密,通过 cors 中间件处理跨域请求,通过 dotenv 管理环境配置。整体数据库配置如下:

```
const express = require('express');           // 引入 Express 框架
const mysql = require('mysql2/promise');       // 引入 MySQL2 的 promise 版本
const bodyParser = require('body-parser');
const cors = require('cors');
const fs = require('fs');
const dotenv = require('dotenv');
const bcrypt = require('bcryptjs'); // 用于密码加密
dotenv.config();

const app = express();
const PORT = 3000;
app.use(cors());
app.use(bodyParser.json());

const multer = require('multer');
const path = require('path');

// 确保上传目录存在
const uploadDir = path.join(__dirname, 'public', 'uploads');
if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true });
}

// 配置 Multer 存储引擎
const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, uploadDir);
  },
  filename: function (req, file, cb) {
```

```

const uniqueSuffix = Date.now() + '_' + Math.round(Math.random() * 1e9);
const ext = path.extname(file.originalname);
cb(null, uniqueSuffix + ext);
}
});
// 创建 multer 实例
const upload = multer({
  storage: storage,
  limits: { fileSize: 10 * 1024 * 1024 } // 限制文件大小为 10MB
});
// 静态资源目录配置
app.use('/uploads', express.static(path.join(__dirname, 'public', 'uploads')));
let connection; // 声明连接变量
// 初始化数据库连接
async function initDb() {
  try {
    connection = await mysql.createConnection({
      host: 'localhost',
      user: 'root',
      password: '*****',
      database: 'web'
    });
    console.log('数据库连接成功');
  } catch (err) {
    console.error('数据库连接失败:', err);
    process.exit(1); // 连接失败时退出应用
  }
}

```

而在本次项目中，一共建立了 8 张表，使用该工具可视化如图 27 所示：

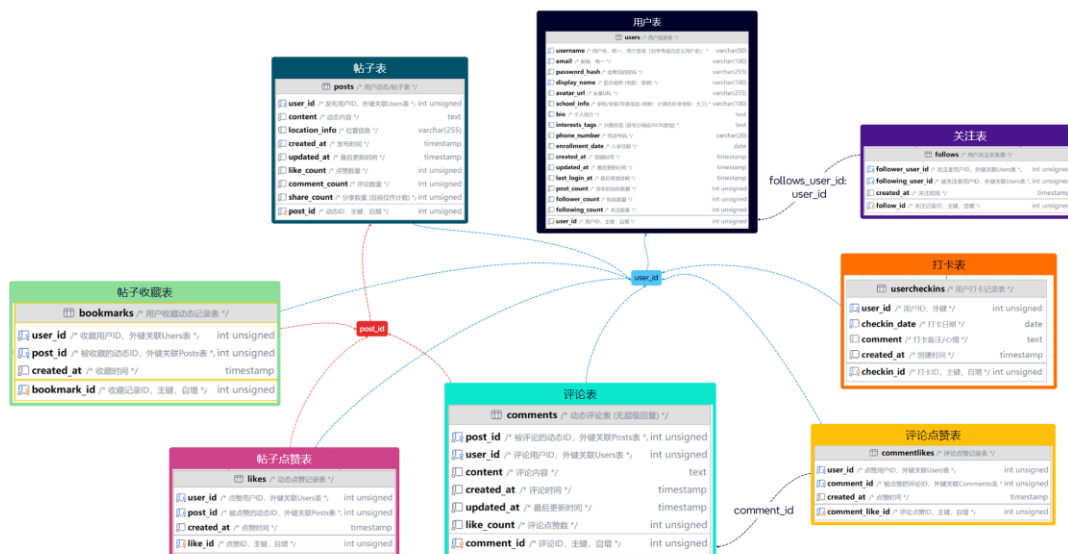


图 27. 数据库表视图

五、项目总结反思

本次实验使用了 HTML、CSS 和 JavaScript 进行了网页设计，脚踏实地从零一直往上堆低的。期间在完成项目时也收获到许多团队开发经验比如团队维护一个文档进行统一操作，才能实现信息共享，同时队长要在群里凝聚队员的定力，督促每一部分的完成，这是保证一个项目完整推进的基础；此外，也获得了许多前端开发经验，比如如何使用开发者工具调试，以及返回 400 是客户端问题，500 是服务端问题等实用性信息。总而言之，本次实验收获颇丰。

诚信承诺：

本人郑重承诺在该课程论文完成的过程中不发生任何不诚信现象，一切不诚信所导致的后果均由本人承担。

Integrity Commitment:

I/We solemnly promise that I/we have not plagiarized during the completion of this course paper, and all the consequences caused by plagiarism will be borne by me/us.

麦子垣 洪子敬

溫高煒

签名拍照附件：

Name and Digital Signature of all students in the group

Date:2025.6.22